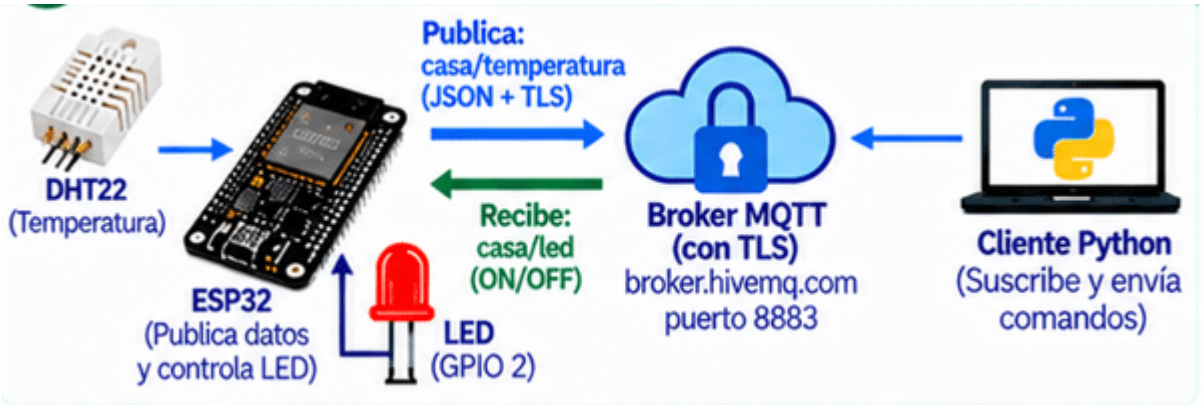
IoT_PracticaDirigida1_RosadoManzur

https://github.com/Manzur68/IoT_PracticaDirigida1_RosadoManzur/tree/main

Objetivo

Implementar un sistema IoT en el que un ESP32 lea la temperatura de un sensor DHT22 y la publique en un broker MQTT seguro (con TLS), y desde un cliente Python se reciban los datos y se pueda enviar un comando para encender o apagar un LED del ESP32

Diagrama del Sistema



Temas y Recursos

- ESP32 con WiFi y DHT22
- MQTT sobre TLS (puerto 8883)
- Certificado CA del broker público
- Librería pubsubClient (ESP32)
- Librerías en Python: paho-mqtt, ssl
- Temas: casa/temperatura y casa/led
- Formato de datos: JSON
- Comando de control: ON / OFF

Preparación del entorno

Paso 1: Configuración en Wokwi

https://wokwi.com/projects/478084860193021953

Conecte los componentes de la siguiente manera:

- **DHT22 (Sensor de Temperatura/Humedad):**
  - `VCC` $\rightarrow$ `3V3` del ESP32
  - `SDA / DATA` $\rightarrow$ `GPIO 15` del ESP32
  - `GND` $\rightarrow$ `GND` del ESP32
  - **LED de Control:**
    - **Ánodo** (Anode) $\rightarrow$ Resistencia de $220,\Omega$ $\rightarrow$ `GPIO 2` del ESP32
    - **Cátodo** (Cathode) $\rightarrow$ `GND` del ESP32

      El codigo en main.py:

```
import network
import time
import ujson
import dht
from machine import Pin
from umqtt.simple import MQTTClient

DHT_PIN = 15
LED_PIN = 2

sensor = dht.DHT22(Pin(DHT_PIN))
led = Pin(LED_PIN, Pin.OUT)
led.value(0)

BROKER = "broker.hivemq.com"
PORT = 8883
TOPIC_PUB = "casa/temperatura"
TOPIC_SUB = "casa/led"
CLIENT_ID = "ESP32_MicroPython_Wokwi"

def conectar_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    if not wlan.isconnected():
```

```
        print("Conectando a WiFi Wokwi: ")
        wlan.connect("Wokwi-GUEST", "")
        while not wlan.isconnected():
            time.sleep(0.5)
            print(".", end="")
    print("\nWiFi Conectado IP:", wlan.ifconfig()[0])

def sub_cb(topic, msg):
    t = topic.decode('utf-8')
    m = msg.decode('utf-8')
    print(f"\n[Mensaje recibido en {t}]: {m}")

    if t == TOPIC_SUB:
        if m == "ON":
            led.value(1)
            print("-LED Encendido")
        elif m == "OFF":
            led.value(0)
            print("-LED Apagado")
conectar_wifi()
print("Conectando al Broker MQTT con TLS:")
client = MQTTClient(
    client_id=CLIENT_ID,
    server=BROKER,
    port=PORT,
    ssl=True,
    ssl_params={'server_hostname': BROKER}
)
client.set_callback(sub_cb)
client.connect()
client.subscribe(TOPIC_SUB)
print(f"Conectado Suscrito a '{TOPIC_SUB}'")

last_pub = 0

while True:
    try:
        client.check_msg()
        if time.time() - last_pub >= 5:
            last_pub = time.time()
            sensor.measure()
            t = sensor.temperature()
            h = sensor.humidity()
            data = {
```

```
                "temp": t,
                "hum": h
            }
            payload = ujson.dumps(data)

            print(f"Publicando en {TOPIC_PUB}: {payload}")
            client.publish(TOPIC_PUB, payload)

    except Exception as e:
        print("Error en el bucle principal:", e)
        time.sleep(1)
```

Paso 2: Configuración del cliente en computadora local

Se levanta el entorno `.venv` y se instala:

```
pip install paho-mqtt
```

El codigo `cliente_desktop.py`:

```
import ssl
import json
import time
import paho.mqtt.client as mqtt

BROKER = "broker.hivemq.com"
PORT = 8883
TOPIC_TEMP = "casa/temperatura"
TOPIC_LED = "casa/led"

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print("\n[OK] Conectado exitosamente al Broker MQTT con TLS.")
        client.subscribe(TOPIC_TEMP)
        print(f"[OK] Suscrito a: {TOPIC_TEMP}\n")
    else:
        print(f"[ERROR] Codigo de conexion fallida: {rc}")

def on_message(client, userdata, msg):
    try:
        payload = msg.payload.decode('utf-8')
```

```
            data = json.loads(payload)
            temp = data.get("temp")
            hum = data.get("hum")
            print(f"\n[DATO RECIBIDO] Temperatura: {temp} °C | Humedad: {hum}
        except Exception:
            print(f"\n[Mensaje Raw] {msg.topic}: {msg.payload.decode('utf-8')

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.tls_set(cert_reqs=ssl.CERT_NONE)
client.tls_insecure_set(True)

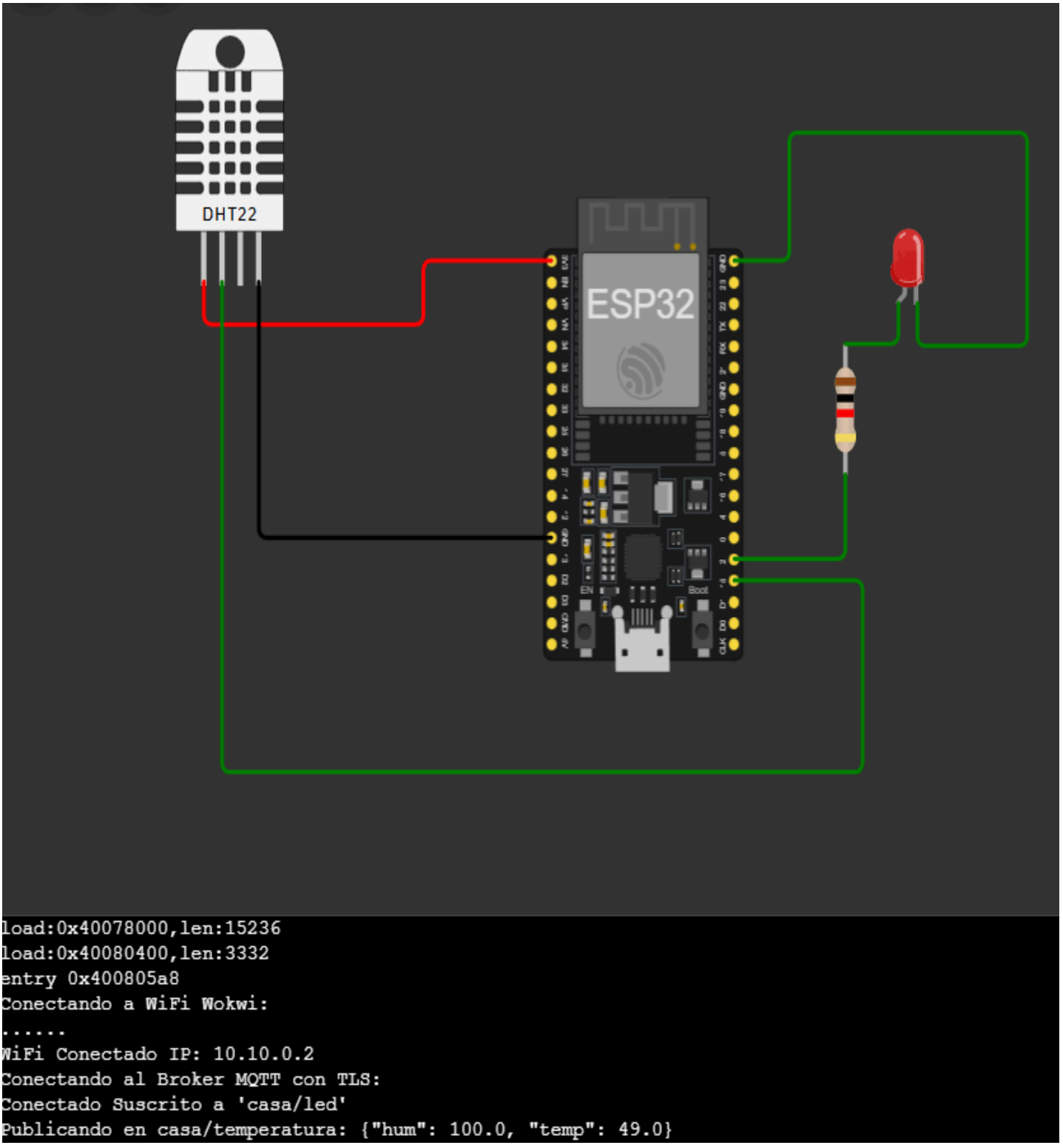
print("Conectando al Broker seguro:")
client.connect(BROKER, PORT, 60)
client.loop_start()

try:
    time.sleep(1)
    print(" Control de LED ESP32")
    while True:
        cmd = input("Comando > ").strip().upper()
        if cmd in ["ON", "OFF"]:
            client.publish(TOPIC_LED, cmd)
            print(f"-> Comando '{cmd}' enviado a '{TOPIC_LED}'")
        elif cmd == "SALIR":
            break
        else:
            print("Comando invalido. Usa 'ON' u 'OFF'")
except KeyboardInterrupt:
    pass
finally:
    client.loop_stop()
    client.disconnect()
    print("Cliente desconectado")
```

Ejecución

1. Conexión segura MQTT + TLS en el ESP32

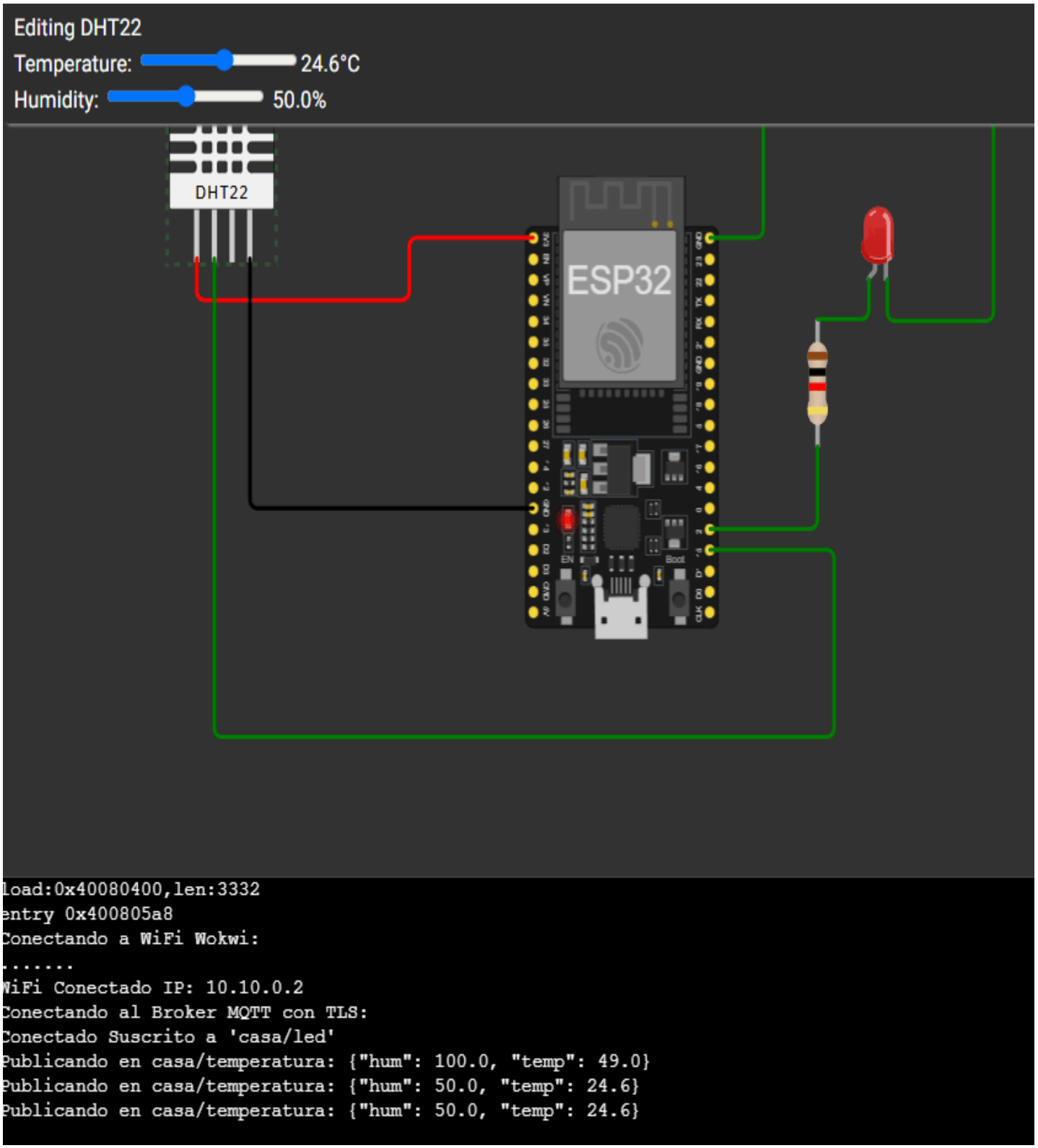
Se configuró el microcontrolador ESP32 en el entorno Wokwi para conectarse a la red WiFi simulada y entablar una comunicación cifrada vía TLS (puerto 8883) con el broker público `broker.hivemq.com`. Se realizó la sincronización horaria previa por NTP para validar de forma correcta el certificado de seguridad.



---

2. Publicación de datos de temperatura (JSON)

El ESP32 realiza lecturas periódicas de temperatura y humedad a través del sensor DHT22 conectado al GPIO 15. Cada 5 segundos, la información es empaquetada en una estructura JSON (`{"temp": xx.x, "hum": xx.x}`) y publicada en el tema `casa/temperatura` empleando el canal seguro TLS.



---

3. Recepción de datos en el cliente Python

Se implementó un script cliente en Python que establece una conexión TLS con `broker.hivemq.com` en el puerto 8883 y se suscribe al tema `casa/temperatura`. El cliente procesa las tramas JSON entrantes y desglosa los valores numéricos para mostrarlos de forma estructurada en la terminal local.

cliente_desktop.py

Modelo-Hibrido > .venv > cliente_desktop.py > on_connect

```python
def on_message(client, userdata, msg):
    try:
        payload = msg.payload.decode('utf-8')
        data = json.loads(payload)
        temp = data.get("temp")
        hum = data.get("hum")
        print(f"\n[DATO RECIBIDO] Temperatura: {temp} °C | Humedad: {hum} %")
    except Exception:
        print(f"\n[Mensaje Raw] {msg.topic}: {msg.payload.decode('utf-8')}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.tls_set(cert_reqs=ssl.CERT_NONE)
client.tls_insecure_set(True)

print("Conectando al Broker seguro:")
client.connect(BROKER, PORT, 60)
client.loop_start()

try:
    time.sleep(1)
    print(" Control de LED ESP32")
    while True:
        cmd = input("Comando > ").strip().upper()
        if cmd in ["ON", "OFF"]:
            client.publish(TOPIC_LED, cmd)
            print(f"-> Comando '{cmd}' enviado a '{TOPIC_LED}'")
        elif cmd == "SALIR":
            break
        else:
            print("Comando invalido. Usa 'ON' u 'OFF'")
except KeyboardInterrupt:
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS D:\Modelo-Hibrido> & d:\Modelo-Hibrido\.venv\Scripts\python.exe d:/Modelo-Hibrido/.venv/cliente_desktop.py
Cliente desconectado
(.venv) PS D:\Modelo-Hibrido> & d:\Modelo-Hibrido\.venv\Scripts\python.exe d:/Modelo-Hibrido/.venv/cliente_desktop.py
Conectando al Broker seguro:

[OK] Conectado exitosamente al Broker MQTT con TLS.
[OK] Suscrito a: casa/temperatura

 Control de LED ESP32
Comando >
[Mensaje Raw] casa/temperatura: 23.80
```

```
(.venv) PS D:\Modelo-Hibrido> & d:\Modelo-Hibrido\.venv\Scripts\python.exe d:/Modelo-Hibrido/.venv/cliente_desktop.py
[Mensaje Raw] casa/temperatura: 23.80

[Mensaje Raw] casa/temperatura: 23.80

[DATO RECIBIDO] Temperatura: 24.6 °C | Humedad: 50.0 %

[Mensaje Raw] casa/temperatura: 23.80

[Mensaje Raw] casa/temperatura: 23.80

[Mensaje Raw] casa/temperatura: 23.80
```

---

4. Envío de comandos y control del LED

Desde la consola interactiva del cliente Python, se enviaron las órdenes `ON` y `OFF` hacia el tema de control `casa/led`. El ESP32 recibe estas peticiones de manera remota y conmuta el estado lógico del GPIO 2 (encendiendo o apagando el LED asignado).

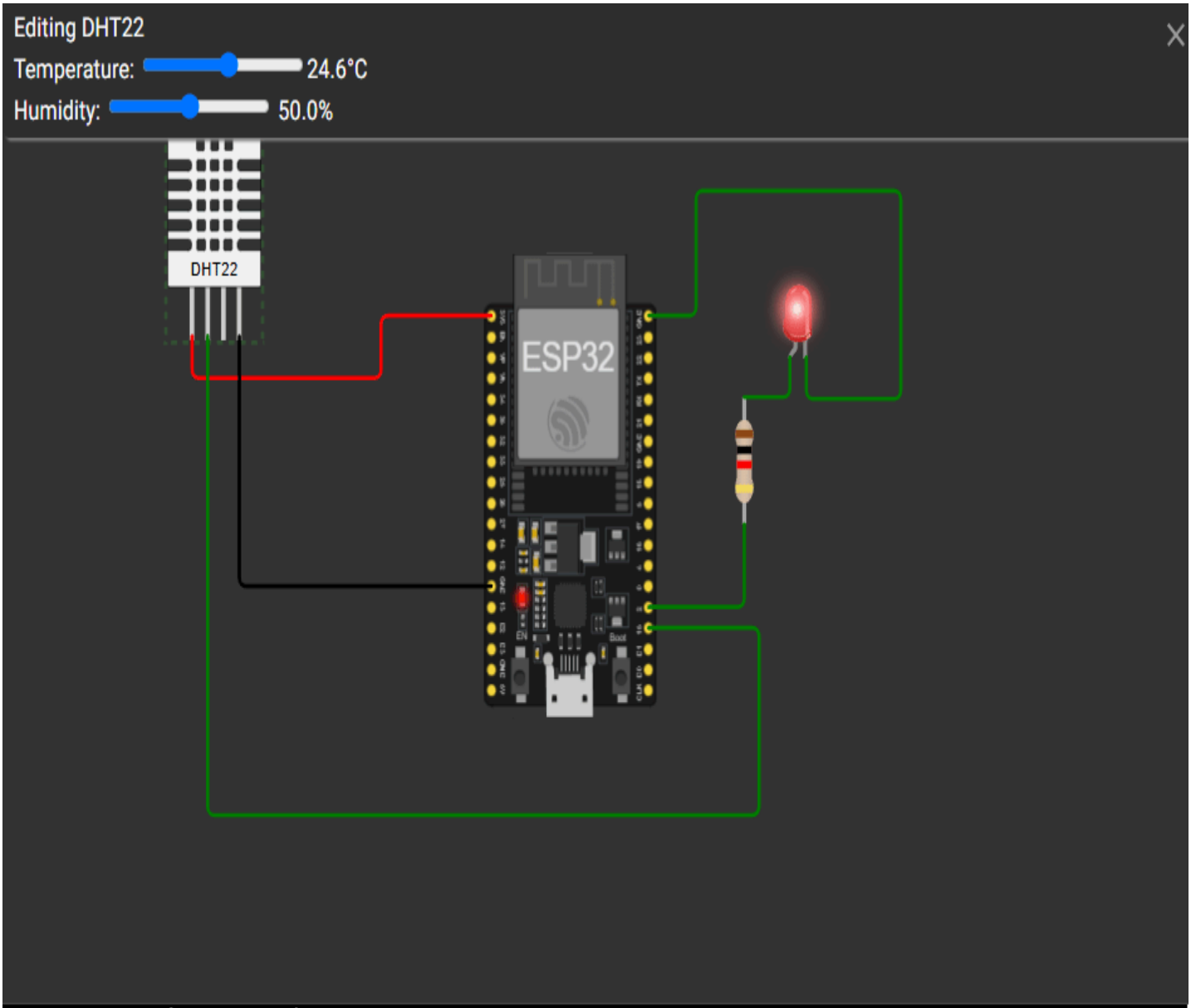
```
(.venv) PS D:\Modelo-Hibrido> & d:\Modelo-Hibrido\.venv\Scripts\python.exe d:/Modelo-Hibrido/.venv/cliente_desktop.py

[Mensaje Raw] casa/temperatura: 23.80

[Mensaje Raw] casa/temperatura: 23.80

[Mensaje Raw] casa/temperatura: 23.90
ON
[Mensaje Raw] casa/temperatura: 23.80

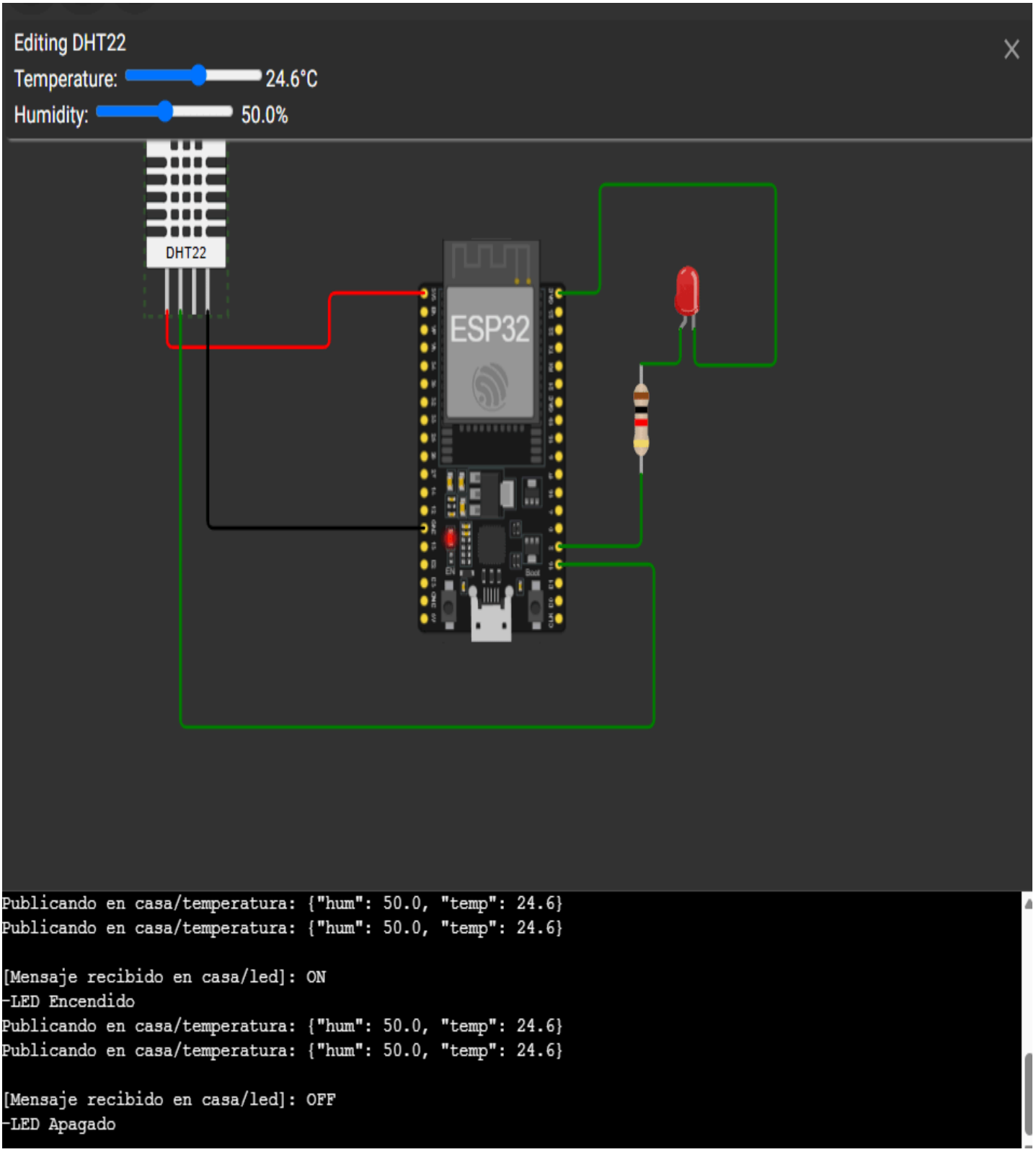
-> Comando 'ON' enviado a 'casa/led'
Comando >
```




```
Conectado Suscrito a 'casa/led'
Publicando en casa/temperatura: {"hum": 100.0, "temp": 49.0}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}

[Mensaje recibido en casa/led]: ON
-LED Encendido
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
```

```
[Mensaje Raw] casa/temperatura: 23.80
OFF
-> Comando 'OFF' enviado a 'casa/led'
Comando >
```



---

5. Funcionamiento general del sistema e integración

Verificación del flujo bidireccional IoT seguro. El sistema mantiene en simultáneo la transmisión remota de telemetría y la recepción de acciones de control sin interrupciones.

main.py ● diagram.json ● ▾

```
32  def sub_cb(topic, msg):
37      if t == TOPIC_SUB:
38          if m == "ON":
41          elif m == "OFF":
42              led.value(0)
43              print("-LED Apagado")
44  conectar_wifi()
45  print("Conectando al Broker MQTT con TLS:")
46  client = MQTTClient(
47      client_id=CLIENT_ID,
48      server=BROKER,
49      port=PORT,
50      ssl=True,
51      ssl_params={'server_hostname': BROKER}
52  )
53  client.set_callback(sub_cb)
54  client.connect()
55  client.subscribe(TOPIC_SUB)
56  print(f"Conectado Suscrito a '{TOPIC_SUB}'")
57
58  last_pub = 0
59
60  while True:
61      try:
62          client.check_msg()
63          if time.time() - last_pub >= 5:
64              last_pub = time.time()
65              sensor.measure()
66              t = sensor.temperature()
67              h = sensor.humidity()
68              data = {
69                  "temp": t,
70                  "hum": h
71              }
72              payload = ujson.dumps(data)
73
74              print(f"Publicando en {TOPIC_PUB}: {payload}")
75              client.publish(TOPIC_PUB, payload)
76
77      except Exception as e:
78          print("Error en el bucle principal:", e)
79          time.sleep(1)
```

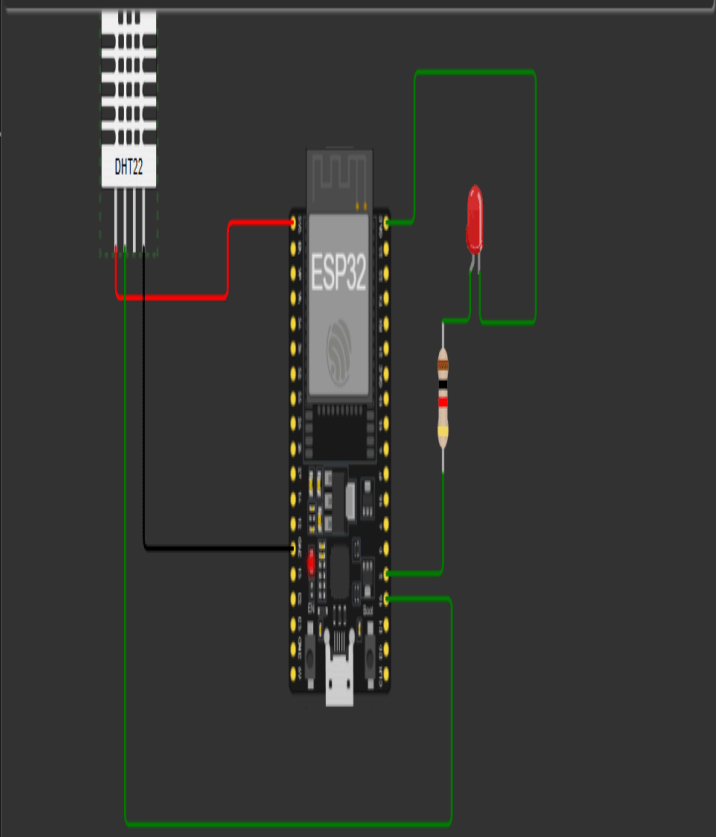
Simulation

00:54.232 54%

Editing DHT22

Temperature: 24.6°C

Humidity: 50.0%




```
[Mensaje recibido en casa/led]: ON
-LED Encendido
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}

[Mensaje recibido en casa/led]: OFF
-LED Apagado
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
Publicando en casa/temperatura: {"hum": 50.0, "temp": 24.6}
```